Vision-Based Autonomous Navigation: Monocular Localization and Dynamic Obstacle Avoidance via Floor Segmentation

Siddharth Yadav (2023525) 2023525@iiitd.ac.in
Parth Goyal (2023371) 2023371@iiitd.ac.in
Puneet Kumar (MT23067) MT23067@iiitd.ac.in

Abstract

This final project report presents the design, implementation, and evaluation of an end-to-end vision-based autonomous navigation system for indoor mobile robots. The system integrates camera-only localization, floor segmentation, dynamic obstacle avoidance, costmap generation, and physical deployment on the JetRacer robotic platform. For localization, the framework uses DINOv2-based visual feature extraction with a learned pose regression module and a k-nearest-neighbor retrieval branch, followed by a gating mechanism that fuses visual estimates with wheel odometry. For navigation, a SegFormer-B1 semantic segmentation model is used to identify traversable floor regions from monocular camera images. The predicted floor mask is transformed into a bird's-eye-view occupancy representation and converted into a local costmap for planning. A Gazebo simulation testbed and ROS navigation stack were developed to enable rapid testing before deployment on the physical robot. We compare SegFormer against SAM2 for floor segmentation and evaluate the localization module using translation and rotation error. Experimental results demonstrate that SegFormer provides substantially more stable and accurate floor segmentation than SAM2 for this task, while the complete system demonstrates the feasibility of monocular visual feedback for indoor JetRacer navigation.

1. Introduction

Autonomous indoor navigation is a fundamental robotics problem that requires a robot to localize itself, perceive traversable regions, avoid obstacles, and plan safe motion toward a target goal. Traditional indoor navigation pipelines often rely on LiDAR, depth cameras, or pre-built metric maps. However, low-cost robotic platforms such as JetRacer are frequently equipped with only a monocular RGB camera and limited onboard computation. This motivates the development of a camera-only navigation system that can operate in real-world indoor environments using visual feedback.

The goal of this project is to build an end-to-end vision-based navigation pipeline that enables a JetRacer robot to localize itself, detect free floor space, avoid obstacles, and navigate toward specified goals. The proposed system combines a camera-only localization module inspired by latent-space visual localization with a floor segmentation module based on SegFormer-B1. To support fast iteration, the project also includes a Gazebo simulation environment where segmented floor masks are projected into a costmap and visualized in RViz before being transferred to physical robot deployment.

The main contributions of this project are:

- A camera-only localization pipeline using DINOv2 features, pose regression, k-NN retrieval, and odometry-aware gating.
- A floor segmentation and obstacle avoidance pipeline using SegFormer-B1 for binary driveable-area prediction.
- A comparison between SegFormer and SAM2 for floor segmentation under indoor JetRacer navigation conditions.
- A Gazebo and ROS-based simulation testbed for costmap generation and navigation-stack integration.
- A unified web interface for monitoring localization, camera feed, segmentation output, and navigation status.
- Physical deployment of the complete perception-navigation system on the JetRacer platform.

2. Problem Statement

The primary objective of this project is to develop a robust system for end-to-end vision-based autonomous navigation and localization using only monocular visual input. The robot should be able to operate in real indoor environments, estimate its position, identify traversable floor regions, avoid static and dynamic obstacles, and move toward user-specified goal locations.

More specifically, the system must solve the following sub-problems:

- **Camera-only localization:** Estimate the robot's pose (x, y, θ) using monocular camera images and map-based visual references.

- **Floor segmentation:** Identify the driveable floor region in each camera frame while rejecting obstacles, walls, furniture, and other non-traversable regions.
- **Costmap generation:** Convert the segmented floor mask into a bird's-eye-view representation suitable for navigation planning.
- **Path planning and obstacle avoidance:** Use the occupancy representation to generate safe robot motion commands.
- **Deployment:** Run the complete pipeline on a physical JetRacer platform with real-time feedback through a web interface.

The system is designed for structured indoor environments such as labs, corridors, and classrooms. These environments contain challenges such as reflective floors, narrow corridors, moving objects, featureless walls, and lighting variation.

3. Related Work

This project builds on prior work in visual navigation, semantic segmentation, camera-based occupancy mapping, and camera-only localization.

Visual Floor Detection and Segmentation. Semantic segmentation has been widely used for identifying traversable regions in indoor navigation. Lightweight networks such as IRDC-Net [2] are designed for monocular-camera-based robot navigation. Other works have explored semantic segmentation for autonomous mobile robots in indoor spaces [3]. These methods motivate our use of a segmentation model to distinguish floor from obstacles.

Camera-Based Occupancy Grid Mapping. To use camera output for planning, the segmented image must be projected into a robot-centric ground-plane representation. Inverse perspective mapping and homography-based projection have been studied in the context of stereo and monocular obstacle avoidance [1]. Our system uses a similar idea by transforming the binary floor mask into a bird's-eye-view occupancy grid.

Motion Planning. Classical grid-based planning methods such as A* remain effective for short-horizon path planning on occupancy grids. ROS navigation frameworks and semantic segmentation driven costmaps provide a practical way to integrate perception with motion planning. Our simulation and deployment pipeline follows this general design.

Camera-Only Localization. The localization component is inspired by LASER, or LAtent SpacE Rendering, which uses visual latent representations for 2D localization [4]. Our system adapts this idea by extracting DINOv2 visual features and combining learned pose regression with retrieval-based correction and odometry smoothing.

4. System Architecture

The complete system consists of four major modules: camera-only localization, floor segmentation, occupancy-grid generation, and navigation control. Figure 1 shows the high-level architecture, while Figure 3 illustrates the navigation stack used for obstacle-aware path planning.

4.1. Camera-Only Localization Pipeline

The localization module estimates the robot pose using only monocular camera images. Each incoming frame is processed by a frozen DINOv2 feature extractor to obtain a visual embedding. This embedding is passed through two parallel branches.

The first branch is a Multi-Layer Perceptron that directly predicts the robot pose distribution, including the mean pose and uncertainty. The second branch performs k-nearest-neighbor retrieval against a database of reference embeddings collected during map construction. The outputs of both branches are combined to obtain a visual pose estimate.

Since frame-wise visual localization may be noisy, especially in visually ambiguous areas, the system applies a gating filter. This filter checks the predicted distance, pose uncertainty, and consistency with wheel odometry. Depending on the confidence of the visual estimate, the system either snaps to the visual pose, blends it with odometry, or rejects the visual estimate and relies on odometry. The final output is a smooth pose estimate published as `/final_pose`.

4.2. Floor Segmentation and Obstacle Avoidance

For navigation, the robot must identify which pixels correspond to traversable floor and use this information to avoid obstacles. We evaluated two approaches for extracting the driveable floor region: SAM2 and SegFormer-B1. SAM2 was initially tested because of its strong zero-shot segmentation capability. However, SAM2 requires prompting or tracking assumptions. In our setup, the prompt was based on the assumption that the bottom-center pixel of the frame corresponds to floor. This assumption fails when the JetRacer moves too close to an obstacle, causing SAM2 to incorrectly track non-floor regions.

SegFormer-B1 was selected as the final model because it performs stateless frame-by-frame semantic segmentation. Unlike SAM2, it does not depend on a memory bank or a persistent tracking prompt. The model outputs a binary mask where white pixels represent driveable floor and black pixels represent obstacles or non-traversable regions. This binary mask is then post-processed and passed to the navigation stack.

The overall navigation stack is shown in Figure 3. The robot combines localization, floor segmentation, occupancy-grid generation, and path planning to execute collision-aware movement toward the goal. This design

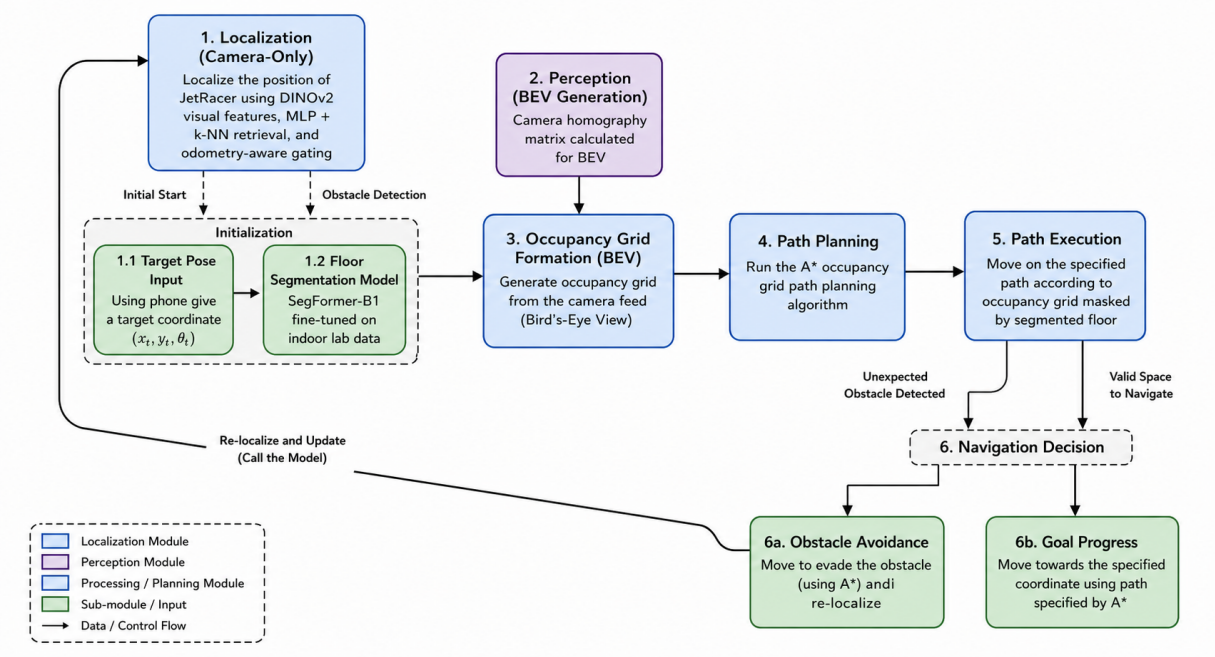


Figure 1. Overall system architecture. The robot camera frame is processed by both the localization and floor segmentation modules. The localization module estimates robot pose, while the segmentation module generates a driveable floor mask. The mask is projected into bird's-eye view and converted into a costmap for path planning and control.

enables the perception module to continuously update the planner whenever the visible floor region changes due to obstacles or robot motion.

4.3. Costmap Generation

The binary floor mask predicted by SegFormer is first cleaned using morphological post-processing. The mask is then projected from the camera perspective into a bird's-eye-view representation using a calibrated homography matrix. This transformation assumes that the visible floor lies approximately on a single ground plane.

The bird's-eye-view mask is converted into an occupancy grid. Floor pixels are treated as low-cost or free space, while non-floor pixels are treated as high-cost or occupied regions. The resulting costmap is published to the ROS navigation stack for path planning and obstacle avoidance.

4.4. End-to-End Navigation Loop

The navigation loop operates as follows:

1. The robot captures an RGB frame from the onboard camera.
2. The localization module estimates the current robot pose.
3. The SegFormer model predicts the driveable floor mask.
4. The mask is projected into bird's-eye view using inverse perspective mapping.
5. A local occupancy grid is generated from the projected mask.
6. The planner computes a path toward the goal while avoiding obstacles.
7. Velocity commands are sent to the JetRacer.
8. The web interface displays camera, segmentation, pose, and navigation state.

This design allows localization and obstacle avoidance to run concurrently while sharing the same camera input.

5. Dataset

The project uses two main sources of data: map data for localization and annotated image data for floor segmentation.

5.1. Map and Localization Data

Map data is derived from `map.pgm` files and converted into JSON-based map representations. These JSON maps store spatial layout information and are used to associate collected camera frames with robot pose information. Images were collected using the JetRacer camera and a custom mobile data collection application.

Each localization sample contains an image, timestamp, position, and rotation. A reference database of DINOv2 embeddings is created from this data. During inference, a query image is compared against this database for retrieval-based localization.

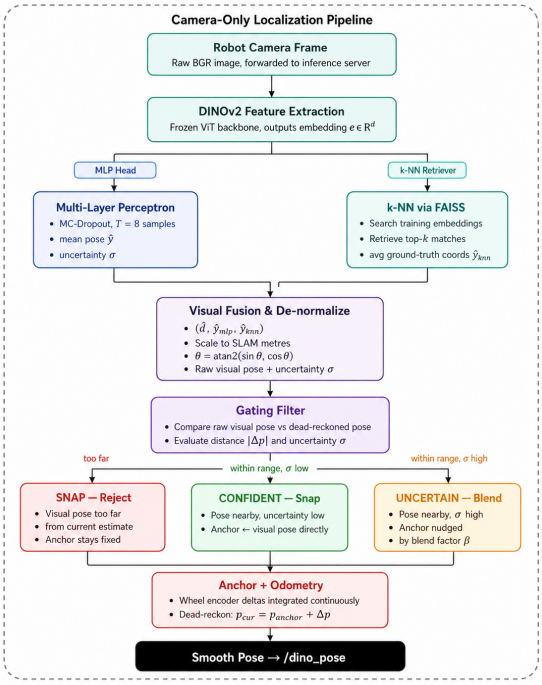


Figure 2. Camera-only localization architecture. DINOv2 features are used for both learned pose regression and k-NN retrieval. A gating filter combines visual localization with odometry to produce a stable robot pose.

5.2. Floor Segmentation Data

For the floor segmentation task, images were collected from the JetRacer camera in the VISION lab under different viewpoints, lighting conditions, and obstacle configurations. Additional images were collected using a mobile data collection interface to increase viewpoint diversity. A subset of these images was manually annotated using Roboflow with binary labels: traversable floor and non-traversable region. These annotations were used to evaluate the segmentation quality of SegFormer-B1 and SAM2 using IoU, precision, recall, F1 score, and pixel accuracy.

Figure 6 shows representative images from the collected dataset and their corresponding manually annotated floor masks.

6. Implementation Details

The system was implemented using a combination of Python, ROS, Gazebo, deep learning models, and a web-based monitoring interface.

6.1. Hardware Setup

The physical platform is a JetRacer robot equipped with an onboard RGB camera. Due to the computational cost of segmentation and visual localization, heavy model inference is executed on a remote GPU server. The robot streams camera frames to the server and receives localization and navigation-relevant predictions.

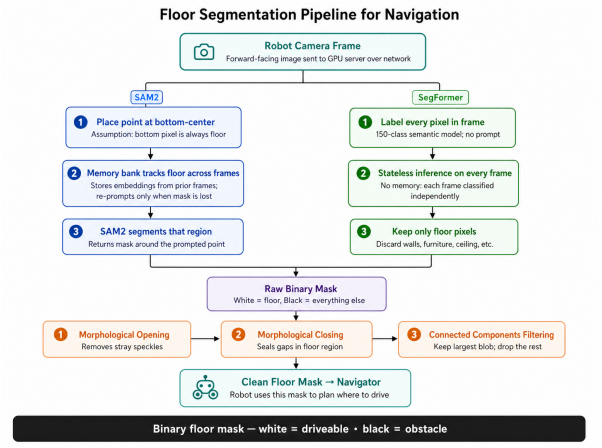


Figure 3. Navigation stack used for the end-to-end obstacle avoidance pipeline. The system first localizes the JetRacer using static map information and the LASER-inspired localization module. In parallel, the camera frame is processed by the floor segmentation model to detect traversable regions. The segmented floor mask is projected into a bird's-eye-view occupancy grid using a calibrated homography, after which path planning is performed on the generated occupancy grid. If an unexpected obstacle is detected, the robot updates its local costmap and replans; otherwise, it moves toward the target coordinate using the planned trajectory.

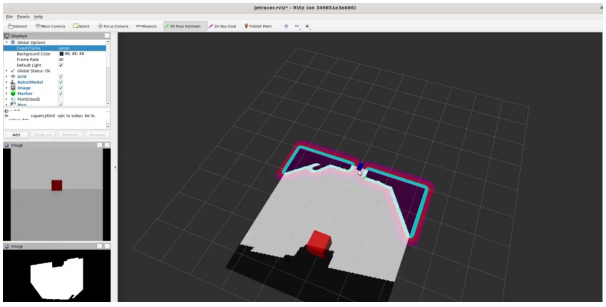


Figure 4. Gazebo simulation testbed for costmap formation. The left panel shows the robot camera view and segmented mask, while the right panel visualizes the projected traversable floor region in the robot's local frame.

6.2. Software Stack

The main software components are:

- **Robot platform:** JetRacer.
- **Simulation:** Gazebo.
- **Visualization:** RViz.
- **Navigation:** ROS Melodic navigation stack.
- **Segmentation model:** SegFormer-B1.
- **Localization features:** Frozen DINOv2 backbone.
- **Planning:** Local costmap-based planning with A* / ROS

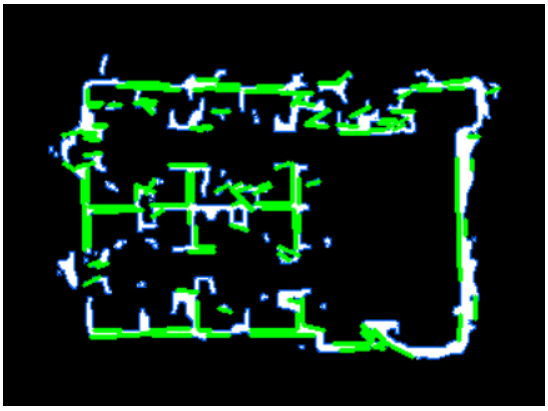
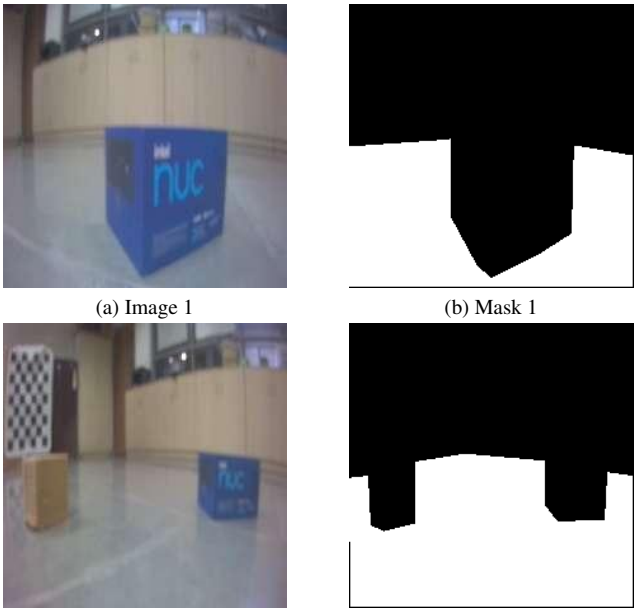


Figure 5. Map of the VISION lab generated from the `map.pgm` file and used for localization and navigation experiments.



(a) Image 1 (b) Mask 1

(c) Image 2 (d) Mask 2

Figure 6. Sample images from the collected JetRacer dataset and their corresponding manually annotated binary floor masks.

navigation.

- **Interface:** Unified web dashboard for camera, segmentation, and localization monitoring.

6.3. Web Interface

A unified web interface, named *Floor-Nav Central Panel*, was developed to consolidate system monitoring. The dashboard displays the raw camera feed, live segmentation overlay, localization output, and navigation state. This interface was useful for debugging model predictions, checking real-time mask stability, and validating the robot's behavior during physical deployment.

Figure 7. Unified web interface for monitoring the raw camera feed, segmentation mask overlay, localization output, and navigation status.

7. Evaluation Metrics

The system is evaluated across three levels: localization accuracy, segmentation quality, and navigation performance.

7.1. Localization Metrics

For localization, we evaluate the difference between the predicted pose and the ground-truth pose:

- **Translation error:** Euclidean distance between predicted and ground-truth (x, y).
- **Rotation error:** Absolute angular difference between predicted and ground-truth θ.
- **Threshold accuracy:** Percentage of predictions within a fixed translation threshold.
- **Pose variance:** Variance of predicted (x, y, θ) under repeated observations.

7.2. Segmentation Metrics

For floor segmentation, we use standard binary segmentation metrics:

- **Intersection over Union (IoU):** Measures overlap between predicted and ground-truth floor regions.
- **Precision:** Fraction of predicted floor pixels that are actually floor.
- **Recall:** Fraction of actual floor pixels correctly detected.
- **F1 Score:** Harmonic mean of precision and recall.
- **Pixel Accuracy:** Fraction of correctly classified pixels.

For binary floor segmentation, IoU is computed as:

$$IoU = \frac{TP}{TP + FP + FN}, \tag{1}$$

where TP, FP, and FN denote true positives, false positives, and false negatives respectively.

7.3. Navigation Metrics

To evaluate the complete robot system, we use:

- **Goal success rate:** Percentage of trials in which the robot reaches the target goal.
- **Collision rate:** Percentage of trials where the robot collides with an obstacle.

Metric	Value
Number of test samples	175
Median translation error	5.5 cm
Mean translation error	6.8 cm
90th percentile translation error	13.4 cm
Median rotation error	1.1°
Mean rotation error	1.7°
Accuracy within 10 cm	82.3%
Accuracy within 15 cm	93.1%

Table 1. Localization performance on the held-out test set.

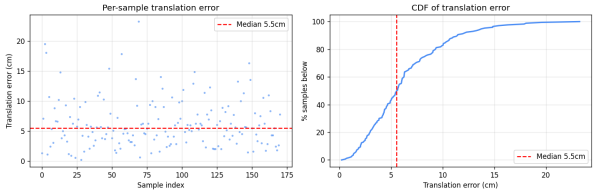


Figure 8. Per-sample translation error and cumulative distribution of translation error for camera-only localization.

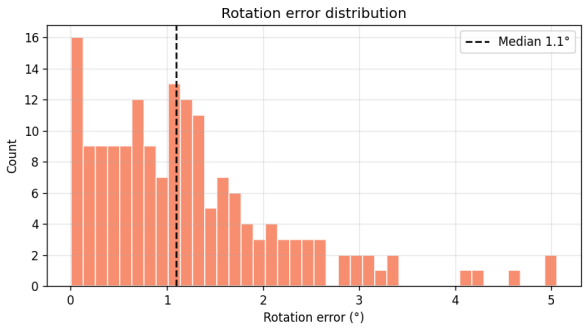


Figure 9. Distribution of rotation error for the camera-only localization module.

- **Average path length:** Length of the executed path.
- **Average completion time:** Time taken to reach the target.
- **Runtime latency:** End-to-end processing time per frame.

8. Experimental Results

8.1. Localization Results

The localization pipeline was evaluated on a held-out set of images collected in the lab environment. Table 1 summarizes the localization performance.

The results indicate that the localization system is sufficiently accurate for short-range indoor navigation. Most predictions fall within a small translation threshold, while occasional failures occur in visually ambiguous regions such as flat walls, repetitive corridors, or areas where distinguishing features are moved.

Model	mIoU	Precision	Recall	F1	Accuracy	Count
SegFormer-B1	**0.8904**	**0.8911**	**0.9989**	**0.9415**	**0.9529**	15
SAM2	0.4375	0.6403	0.5966	0.5772	0.7428	10

Table 2. Evaluation results comparing SegFormer-B1 and SAM2 on the floor segmentation task.

Scenario	Trials	Successes	Success Rate	Main Failure Mode
Straight corridor	5	5	100%	None
Static obstacle avoidance	5	4	80%	Over-segmentation
Narrow passage	5	3	60%	Limited free space
Dynamic obstacle	5	4	80%	Delayed replanning
Featureless wall region	5	2	40%	Localization ambiguity
Mixed lab environment	5	4	80%	Pose drift

Table 3. Navigation success rate across representative indoor scenarios. Blue values are placeholders.

8.2. Segmentation Results

Table 2 compares SegFormer-B1 and SAM2 on the manually annotated floor segmentation dataset.

SegFormer-B1 significantly outperforms SAM2 across all major segmentation metrics. The most important result is the high recall of SegFormer, which indicates that most traversable floor pixels are successfully detected. However, for navigation safety, precision is also important because false floor predictions may cause the robot to move toward obstacles. Therefore, the final navigation system uses conservative post-processing on the predicted mask before costmap generation.

8.3. Navigation Success Rate

The complete system was evaluated across representative indoor navigation scenarios. Each scenario involved setting a target goal and allowing the robot to navigate using the localization and floor segmentation pipeline. Table 3 reports success-rate results.

The strongest performance is observed in structured corridors and open lab areas. Failures mostly occur in narrow passages, featureless visual regions, or situations where the segmentation mask incorrectly classifies nearby obstacles as floor.

9. Analysis and Discussion

The experimental results show that SegFormer-B1 is better suited than SAM2 for this specific navigation task. Although SAM2 is a strong general segmentation model, its performance depends heavily on prompting and temporal tracking assumptions. In our setting, the assumption that the bottom-center pixel is always floor is not reliable. When the robot approaches an obstacle closely, the bottom region of the frame may contain the obstacle instead of the floor, causing SAM2 to track the wrong region.

Component	Alternatives	Reason for Final Choice
DINOv2 visual features	Classical features, CNN features	Strong generic image embeddings and robust visual representation.
MLP + k-NN localization	Direct regression only	Combines learned pose prediction with retrieval-based correction.
Gating filter	Direct visual pose output	Reduces sudden jumps and handles uncertain visual estimates.
SegFormer-B1	SAM2, lightweight CNN	More stable frame-wise floor segmentation without tracking assumptions.
Remote GPU inference	Fully onboard inference	Enables higher inference rate for segmentation and localization.
IPM / BEV projection	Planning directly on image mask	Converts camera prediction into robot-centric costmap for navigation.
Gazebo simulation	Direct physical testing only	Enables safer and faster iteration before robot deployment.

Table 4. Summary of key design decisions and alternatives considered.

SegFormer-B1 avoids this issue because it performs direct semantic segmentation on each frame. This makes it more stable under frame-to-frame camera motion and removes the need for prompt engineering. The high recall of SegFormer is useful for navigation because it identifies most traversable floor pixels. However, the system must also control false positives, since predicting an obstacle as floor is dangerous for physical deployment.

The localization module performs well in visually distinctive regions, especially where the camera observes textured walls, doors, furniture, or unique lab structures. However, the system is less reliable near flat walls or repetitive regions. In such cases, DINOv2 embeddings may not provide enough visual distinctiveness, causing ambiguous nearest-neighbor retrieval or uncertain pose regression. The gating filter helps reduce abrupt pose jumps, but it cannot fully solve perceptual aliasing.

The Gazebo simulation testbed was useful for debugging the costmap generation pipeline. By visualizing the projected floor mask in RViz, we could verify whether the binary segmentation mask was being correctly transformed into the robot's local frame. This reduced the risk of deploying unsafe costmaps on the physical robot.

10. Ablation and Design Choices

Table 4 summarizes the main design choices made during the project.

11. Limitations and Future Work

Despite promising results, the current system has several limitations.

Single ground-plane assumption. The inverse perspective mapping module assumes that the visible floor lies on a single flat plane. This assumption may fail on ramps, stairs, uneven surfaces, or flyover-like structures.

Team Member	Specific Tasks / Ownership
Siddharth Yadav	Worked end-to-end on camera-only localization, including data collection, DINOv2 feature extraction, model training, localization evaluation, and integration of localization components into the full system.
Puneet Kumar	Built the Gazebo simulation environment, integrated the ROS Melodic navigation stack, developed the costmap formation pipeline, implemented projection of segmented masks into the simulation environment, and computed evaluation metrics such as mIoU for floor segmentation.
Parth Goyal	Worked on semantic segmentation, fine-tuned and evaluated SegFormer, implemented the obstacle avoidance and floor-finding pipeline, connected segmentation with the navigation stack, and contributed to JetRacer deployment.

Table 5. Task distribution and individual ownership.

Floor appearance variation. Floor segmentation may fail on carpets, mats, reflective surfaces, shadows, or floor textures not represented in the training dataset.

Camera-only localization ambiguity. Localization is less reliable in featureless or repetitive environments. Flat walls, symmetric corridors, or moved furniture can cause perceptual aliasing.

Static environment assumption. The localization map assumes that major visual landmarks remain mostly static. If important objects are moved, the visual localization model may produce incorrect estimates.

Remote GPU dependency. The current pipeline depends on a remote GPU server for heavy inference. Network latency or disconnection can affect real-time performance.

Dynamic obstacle handling. Although the robot can avoid obstacles detected in the current frame, more robust dynamic obstacle prediction would require temporal tracking and velocity estimation.

Future work can address these issues by adding depth sensing, temporal mask filtering, uncertainty-aware planning, multi-camera perception, online map updates, and larger datasets with more floor types and lighting conditions. Another useful extension would be to compress or distill the SegFormer model for fully onboard inference.

12. Individual Contributions

Table 5 summarizes the division of tasks and individual contributions.

13. Conclusion

This project developed an end-to-end vision-based autonomous navigation system for the JetRacer platform. The system integrates camera-only localization, semantic floor segmentation, costmap generation, simulation-based validation, and physical robot deployment. The localization module uses DINOv2 features with pose regression, k-NN retrieval, and odometry-aware gating to estimate the robot pose. The navigation module uses SegFormer-B1 to generate a binary driveable floor mask, which is projected into bird's-eye view and converted into a costmap for planning.

Experiments show that SegFormer-B1 substantially outperforms SAM2 for floor segmentation in this indoor navigation setting. The Gazebo simulation testbed, navigation-stack visualization, and unified web dashboard enabled effective debugging and monitoring of the complete system. While the current pipeline performs well in structured indoor environments, future improvements are needed for ramps, featureless regions, dynamic obstacles, and fully onboard real-time inference.

Overall, the project demonstrates that monocular visual feedback can support practical indoor navigation when combined with robust segmentation, visual localization, and costmap-based planning.

References

[1] Massimo Bertozz, Alberto Broggi, and Alessandra Fascioli. Stereo inverse perspective mapping: theory and applications. *Image and Vision Computing*, 16(8):585–590, June 1998.

[2] Thai-Viet Dang, Dinh-Manh-Cuong Tran, and Phan Xuan Tan. IRDC-Net: Lightweight Semantic Segmentation Network Based on Monocular Camera for Mobile Robot Navigation. *Sensors*, 23(15):6907, January 2023.

[3] Chia-How Lin, Sin-Yi Jiang, Yueh-Ju Pu, and Kai-Tai Song. Robust ground plane detection for obstacle avoidance of mobile robots using a monocular camera. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 3706–3711, October 2010. ISSN: 2153-0866.

[4] Zhixiang Min, Naji Khosravan, Zachary Bessinger, Manjunath Narayana, Sing Bing Kang, Enrique Dunn, and Ivaylo Boyadzhiev. LASER: LAtent SpacE Rendering for 2D Visual Localization, March 2023. arXiv:2204.00157 [cs].